

Security Scan Report

Target: C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab

Scan Date: May 05, 2026 at 19:03 UTC

Scan ID: 8e7d07ab-09c5-438e-9af0-b5c2980a8a15

0 CRITICAL

2 HIGH

9 MEDIUM

2 LOW

Executive Summary

The recent security scan of your application at C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab identified several critical vulnerabilities that pose significant risks to your system's integrity and data security. The most pressing issues are the use of insecure deserialization in app.py:282, which is susceptible to attacks due to the misuse of a vulnerable library, and an SQL injection vulnerability at app.py:68, where user-controlled inputs can be exploited to manipulate database queries. These vulnerabilities could lead to unauthorized access, data leakage, or system compromise. Immediate attention is required to mitigate these risks by updating the deserialization library and properly sanitizing user inputs in SQL queries.

Scan Statistics

Metric	Value
Total Raw Findings	38
After Deduplication	13
Critical Findings	0
High Findings	2
Medium Findings	9
Low Findings	2
Scan Duration	197.4s

Detailed Findings

[HIGH] #1: Insecure Deserialization in C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab\app.py:282

Risk Score	0.6076
Affected Location	C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab\app.py:282
Vulnerability Type	INSECURE_DESERIALIZATION
CWE ID	CWE-502: Deserialization of Untrusted Data
Detected By	semgrep

Explanation

The vulnerability in question arises from the use of pickle, an insecure deserialization library, to handle user input. Pickle can execute arbitrary Python code because it allows for the deserialization of untrusted data into objects, which can lead to a Remote Code Execution (RCE) attack. In this particular case, user input is decoded using base64 and then passed directly to pickle.loads(), bypassing any necessary validation or sanitization that could prevent malicious serialization payloads fr

Code or HTTP Excerpt

```
280 |         try:
281 |             decoded = base64.b64decode(payload)
>>> 282 |             session_object = pickle.loads(decoded)
283 |             return jsonify({"status": "restored", "object": repr(session_object)})
284 |         except Exception as exc:
```

Business Impact

If exploited, an attacker could gain unauthorized access to sensitive user data or execute arbitrary code on the server, leading to significant financial loss and damage to the organization's reputation.

Fix Recommendation

Rule-based recommendation: Validate and whitelist deserialized types. Avoid using pickle or yaml.load() with untrusted input. Use yaml.safe_load() or JSON for data interchange. Implement integrity checks on serialized data.

LLM recommendation: Replace the use of pickle with a safer alternative such as JSON for serialization and deserialization. Update line 282 in app.py to: session_object = json.loads(decoded.decode('utf-8'))

[MEDIUM] #2: Unknown in C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab\app.py:70

Risk Score	0.5390
Affected Location	C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab\app.py:70
Vulnerability Type	UNKNOWN
CWE ID	CWE-704: Incorrect Type Conversion or Cast
Detected By	semgrep

Explanation

The detected vulnerability in the provided code snippet involves the use of user input to manually construct a SQL query string. This practice, known as 'SQL injection', poses significant risks because it allows an attacker to manipulate the database queries through crafted inputs. By directly incorporating user-provided email and password into the SQL query without proper sanitization or parameterization, any malicious data entered by an attacker could alter the intended operation of the query,

Code or HTTP Excerpt

```
68 |         email = request.form.get("email", "")
69 |         password = request.form.get("password", "")
>>> 70 |         query = f"SELECT * FROM users WHERE email = '{email}' AND password = '{password}'"
71 |         try:
72 |             conn = connect()
```

Business Impact

Exploiting this vulnerability could lead to unauthorized access of sensitive user data, potentially resulting in severe financial loss and damage to the bank's reputation.

Fix Recommendation

Rule-based recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

LLM recommendation: Replace the line 70 with: `query = "SELECT * FROM users WHERE email = ? AND password = ?"` `params = (email, password)` `cursor.execute(query, params)`

[HIGH] #3: Sql Injection in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:68

Risk Score	0.5170
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:68
Vulnerability Type	SQL_INJECTION
CWE ID	CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
Detected By	semgrep

Explanation

The vulnerability in question arises from the direct inclusion of user-controlled input (email and password) into an SQL query string. This practice, known as 'SQL injection,' allows an attacker to manipulate the database query by injecting malicious SQL code through these parameters. If successful, this could lead to unauthorized access or exposure of sensitive information stored in the database. Specifically, if a user inputs a specially crafted email address that includes SQL syntax (e.g., ';

Code or HTTP Excerpt

```
66 |         error = None
67 |         if request.method == "POST":
>>> 68 |             email = request.form.get("email", "")
69 |             password = request.form.get("password", "")
70 |             query = f"SELECT * FROM users WHERE email = '{email}' AND password = '{password}'"
```

Business Impact

Exploiting this vulnerability could lead to significant financial loss and damage the company's reputation by exposing sensitive user data.

Fix Recommendation

Rule-based recommendation: Use parameterized queries instead of string concatenation. In Python, use placeholder syntax: `cursor.execute('SELECT * FROM users WHERE id = ?', (user_id,))`. For ORMs, use the built-in query builder instead of raw SQL.

LLM recommendation: Replace line 68 with: `python email = request.form.get("email", "") password = request.form.get("password", "") query = "SELECT * FROM users WHERE email = %s AND password = %s" params = (email, password) db_cursor.execute(query, params)```

[MEDIUM] #4: Xss in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\admin.html:22

Risk Score	0.4747
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\admin.html:22
Vulnerability Type	XSS
CWE ID	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
Detected By	semgrep

Explanation
The vulnerability in question arises from the use of the '| safe' filter in a Flask template, which explicitly disables automatic HTML escaping for the content within this segment. This means that any user input rendered here can potentially be interpreted as raw HTML, thus allowing for cross-site scripting (XSS) attacks. An attacker could inject malicious JavaScript code into the page, which would then execute when other users access the same page or interact with elements on the page.

Code or HTTP Excerpt

```
20 |         <button class="btn primary" type="submit">Run</button>
21 |     </form>
>>> 22 |     <pre class="console">{{ command_output|safe }}</pre>
23 | </div>
24 | <div class="card">
```

Business Impact
Exploiting this vulnerability could lead to unauthorized access to sensitive information displayed in the browser, potentially allowing an attacker to steal user credentials or perform actions on behalf of authenticated users.

Fix Recommendation
Rule-based recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

LLM recommendation: Remove the '| safe' filter to re-enable automatic HTML escaping for the content within this segment. Update the line as follows: `html <pre class="console">{{ command_output }}</pre>```

[MEDIUM] #5: Unknown in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:10

Risk Score	0.4508
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:10
Vulnerability Type	UNKNOWN
CWE ID	CWE-611: Improper Restriction of XML External Entity Reference
Detected By	semgrep

Explanation
The vulnerability in question arises from the use of the native Python ``xml`` library, which is known to be vulnerable to XML External Entity (XXE) attacks. This type of attack exploits the way the XML parser handles external entities, allowing for the extraction of confidential data or even causing a denial of service by triggering large memory allocations within the application's context. An attacker can exploit this vulnerability by crafting an XML document that references an external entity,

Code or HTTP Excerpt

```
8 | import subprocess
9 | import urllib.request
>>> 10 | import xml.etree.ElementTree as ET
11 | from pathlib import Path
12 | from urllib.parse import unquote, urlparse
```

Business Impact
If exploited, an attacker could gain unauthorized access to sensitive data within the application or system hosting it, leading to significant financial and reputational damage.

Fix Recommendation
Rule-based recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

LLM recommendation: Replace the use of ``xml.etree.ElementTree`` with a more secure alternative such as ``defusedxml``. This can be done by changing line 10 in the file `app.py` to import `ElementTree` from `defusedxml` instead: ````python from defusedxml.ElementTree import ElementTree as ET ````

[MEDIUM] #6: Open Redirect in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:87

Risk Score	0.4508
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:87
Vulnerability Type	OPEN_REDIRECT
CWE ID	CWE-601: URL Redirection to Untrusted Site ('Open Redirect')
Detected By	semgrep

Explanation
The vulnerability in question arises from the use of user input (specifically, the 'next' parameter from a query string) being directly passed to the `redirect()` function without proper validation or sanitization. This allows an attacker to manipulate the URL that the application redirects to, potentially directing users to malicious or unintended sites. An open redirect vulnerability is particularly dangerous because it bypasses traditional security measures such as same-origin policy checks and

Code or HTTP Excerpt

```
85 | @app.route("/logout")
```

```
86 | def logout():
>>> 87 |     next_url = request.args.get("next", url_for("index"))
      88 |     response = redirect(next_url)
      89 |     response.delete_cookie("bank_uid")
```

Business Impact

If exploited, an attacker could trick authenticated users into visiting a malicious site after logging out, potentially leading to unauthorized access to sensitive information or further exploitation of other vulnerabilities.

Fix Recommendation

Rule-based recommendation: Validate redirect URLs against a whitelist of allowed domains. Use relative paths instead of full URLs for internal redirects. Reject any redirect URLs pointing to external domains.

LLM recommendation: Replace the line 87 with: `next_url = request.args.get('next', url_for('index'))`
`parsed_url = urlparse(next_url)` if `parsed_url.netloc != 'your-site.com': next_url = url_for('index')` `response = redirect(next_url)`

[MEDIUM] #7: Unknown in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:313

Risk Score	0.4508
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:313
Vulnerability Type	UNKNOWN
CWE ID	CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
Detected By	semgrep

Explanation

The vulnerability in question arises from the use of `subprocess.run` with `shell=True`, which allows for command injection. In this specific case, user input (obtained via `request.args.get('cmd', 'status')`) is directly passed to `subprocess.run` without any sanitization or validation. This means that an attacker can provide arbitrary commands to be executed by the system, potentially leading to unauthorized access, data leakage, and other malicious activities. For example, they could execute

Code or HTTP Excerpt

```
311 |     cmd = request.args.get("cmd", "status")
312 |     shell_cmd = f"echo Reconciled ledger for {cmd}"
>>> 313 |     proc = subprocess.run(shell_cmd, shell=True, capture_output=True, text=True)
      314 |     summary = fetch_one(
      315 |         "SELECT COUNT(*) AS users, (SELECT COUNT(*) FROM accounts) AS accounts, (SELECT COUNT(*)
```

Business Impact

If exploited, an attacker could execute arbitrary commands on the server, leading to data theft and potentially causing significant financial losses or legal repercussions for the organization.

Fix Recommendation

Rule-based recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

LLM recommendation: Replace line 313 with: `proc = subprocess.run(['echo', 'Reconciled ledger for', cmd], capture_output=True, text=True)` This change ensures that the command is executed as a list of arguments rather than being interpreted as a shell command, thus mitigating the risk of command

injection.

[MEDIUM] #8: Xss in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\account.html:24

Risk Score	0.4042
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\account.html:24
Vulnerability Type	XSS
CWE ID	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
Detected By	semgrep

Explanation
The vulnerability in question arises from the use of the '| safe' filter within a Flask template, which explicitly disables HTML autoescaping for the rendered content. This means that any user input passed to the 'tx.counterparty' or 'tx.note' variables is not escaped and can contain JavaScript code, potentially leading to a cross-site scripting (XSS) attack. An attacker could inject malicious scripts that are executed in the context of the victim's browser, allowing for session hijacking, data

Code or HTTP Excerpt

```
22 |         {% for tx in transactions %}
23 |         <tr>
>>> 24 |             <td>{{ tx.counterparty|safe }}</td>
25 |             <td>{{ '%.2f'|format(tx.amount) }}</td>
26 |             <td>{{ tx.note|safe }}</td>
```

Business Impact
If exploited, an attacker could execute arbitrary JavaScript on the client side, leading to unauthorized access to sensitive information stored in cookies or session tokens, which could result in significant financial loss or legal repercussions for the organization.

Fix Recommendation
Rule-based recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

LLM recommendation: Replace '| safe' with '| escape' to ensure HTML autoescaping is enabled again. The corrected line should look like this: <td>{{ tx.counterparty|escape }}</td>. This change ensures that any potentially malicious code in user input is escaped and rendered harmless.

[MEDIUM] #9: Xss in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\dashboard.html:61

Risk Score	0.4042
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\dashboard.html:61
Vulnerability Type	XSS
CWE ID	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
Detected By	semgrep

Explanation

The detected vulnerability in the Flask template at line 61 is a cross-site scripting (XSS) issue. The '| safe' filter used in the Jinja2 templating engine allows raw HTML to be rendered directly within the template, even if it contains user input. This can lead to an XSS attack where malicious scripts are executed in the victim's browser when they view the page containing this content. An attacker could exploit this by injecting JavaScript code that steals cookies or performs other actions on b

Code or HTTP Excerpt

```
59 |         <td>{{ tx.counterparty }}</td>
60 |         <td>{{ '%.2f'|format(tx.amount) }}</td>
>>> 61 |         <td>{{ tx.note|safe }}</td>
62 |     </tr>
63 |     {% endfor %}
```

Business Impact

Exploiting this vulnerability could lead to unauthorized access to sensitive user data, potentially resulting in financial loss or a significant compromise of trust in the system.

Fix Recommendation

Rule-based recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

LLM recommendation: Remove the '| safe' filter to enable automatic escaping of HTML characters within the 'tx.note' variable. Update the line as follows: 59 | <td>{{ tx.counterparty }}</td> 60 | <td>{{ '%.2f'|format(tx.amount) }}</td> >>> 61 | <td>{{ tx.note }}</td> 62 | </tr> 63 | {% endfor %}

[MEDIUM] #10: Xss in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\reflect.html:7

Risk Score	0.4042
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\reflect.html:7
Vulnerability Type	XSS
CWE ID	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
Detected By	semgrep

Explanation

The vulnerability in the provided Flask template snippet is a cross-site scripting (XSS) issue caused by the use of the '| safe' filter on user inputs. This filter allows raw HTML to be rendered directly within the template, which can lead to XSS attacks if untrusted users input malicious scripts. An attacker could exploit this vulnerability by injecting JavaScript code in the 'name' or 'message' parameters through a crafted URL query string, such as '/reflect?name=<script>alert('XSS')</script>&

Code or HTTP Excerpt

```
5 |     <h1>Customer Notice Board</h1>
6 |     <p class="subtle">This page intentionally renders query params unsafely.</p>
>>> 7 |     <div class="alert">Hello {{ name|safe }}, message: {{ message|safe }}</div>
8 |     <form method="get" class="form inline" action="/reflect">
9 |         <label>Name<input type="text" name="name" value="{{ name|safe }}"></label>
```

Business Impact

Exploiting this vulnerability could lead to unauthorized access to sensitive user data, potentially resulting in significant financial loss and damage to the bank's reputation.

Fix Recommendation

Rule-based recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

LLM recommendation: Replace '| safe' with a default HTML escaping filter like '| e' in the affected template lines. This ensures that any user input is properly escaped before being rendered, preventing XSS attacks. Specifically, modify line 7 from '<div class="alert">Hello {{ name|safe }}, message: {{ message|safe }}</div>' to '<div class="alert">Hello {{ name|e }}, message: {{ message|e }}</div>'.

[MEDIUM] #11: Xss in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\support.html:23

Risk Score	0.4042
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\support.html:23
Vulnerability Type	XSS
CWE ID	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
Detected By	semgrep

Explanation

The vulnerability in question is a cross-site scripting (XSS) flaw present in the Flask template file 'support.html' at line 23. The code snippet indicates that user input within the ticket.body variable is being rendered without proper HTML escaping, thanks to the use of the '| safe' filter. This filter allows any embedded HTML or JavaScript code to be executed by a malicious user in the context of the victim's browser. An attacker could inject arbitrary JavaScript through this vulnerability, p

Code or HTTP Excerpt

```
21 |         <td>{{ ticket.name }}</td>
22 |         <td>{{ ticket.subject }}</td>
>>> 23 |         <td>{{ ticket.body|safe }}</td>
24 |     </tr>
25 |     {% endfor %}
```

Business Impact

Exploiting this vulnerability could lead to unauthorized access to sensitive information, manipulation of web page content, and potential theft of user session cookies or other authentication tokens.

Fix Recommendation

Rule-based recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

LLM recommendation: Remove or replace the '| safe' filter with a proper escaping mechanism to ensure that any HTML or JavaScript code within ticket.body is rendered safely. The corrected line should look like this: <td>{{ ticket.body | e }}</td>. This will prevent the direct execution of user-supplied content and mitigate the XSS risk.

[LOW] #12: Unknown in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:248

Risk Score	0.3384
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:248
Vulnerability Type	UNKNOWN
CWE ID	CWE-939: Improper Authorization in Handler for Custom URL Scheme
Detected By	semgrep

Explanation
The vulnerability in question arises from the use of user-controlled input (specifically, a URL) directly in an 'urllib.request.urlopen' call without proper validation or sanitization. This allows for potential manipulation by an attacker to read arbitrary files on the system through the 'file:/' scheme supported by urllib. If exploited, an attacker could craft a malicious request that targets local files instead of remote servers, potentially leading to unauthorized data exposure or other secu

Code or HTTP Excerpt

```
246 | def fetch_remote():
247 |     target = request.args.get("url", "http://127.0.0.1:5000/")
>>> 248 |     with urllib.request.urlopen(target, timeout=3) as handle:
249 |         body = handle.read(3000).decode("utf-8", errors="replace")
250 |     return Response(body, mimetype="text/plain")
```

Business Impact
If exploited, this vulnerability could lead to unauthorized access to sensitive files on the system, potentially compromising the confidentiality and integrity of the data stored in those files.

Fix Recommendation
Rule-based recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.
LLM recommendation: Replace 'urllib.request.urlopen' with a safer method that validates and sanitizes user input more rigorously, such as using Python's 'requests' library which provides better security features for handling external requests. Specifically, modify line 248 to use 'requests.get' instead of 'urllib.request.urlopen'. Here is the fix: import urllib.request import requests def fetch_remote(): target = request.args.get("url", "http://127.0.0.1:5000/") # Use requests library for safer handling of

[LOW] #13: Unknown in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:369

Risk Score	0.2820
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:369
Vulnerability Type	UNKNOWN
CWE ID	CWE-489: Active Debug Code
Detected By	semgrep

Explanation

The detected vulnerability in the Flask application involves running the app with the 'debug=True' flag enabled, which is not suitable for production environments. When debug mode is enabled in a Flask application, it includes several features that are useful during development but can expose sensitive information and internal server details to users, potentially leading to unauthorized access or data leakage. Specifically, enabling debug mode makes the app more vulnerable to attacks because it

Code or HTTP Excerpt

```
367 |  
368 | if __name__ == "__main__":  
>>> 369 |     app.run(host="127.0.0.1", port=5001, debug=True)
```

Business Impact

Exploiting this vulnerability could lead to unauthorized access and exposure of sensitive information, damaging trust in the application and potentially resulting in significant financial or reputational losses for the organization.

Fix Recommendation

Rule-based recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

LLM recommendation: Replace line 369 with: `app.run(host='127.0.0.1', port=5001)`

Appendix

Deduplication Statistics

Total raw findings: 38 | After deduplication: 13 | Reduction: 66%

Scan Metadata

Scan ID	8e7d07ab-09c5-438e-9af0-b5c2980a8a15
Target	C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab
Timestamp	2026-05-05T19:03:44.979156+00:00
Duration	197.4s
Scanners Used	semgrep
Scanner Versions	semgrep=1.159.0

Raw Findings Data (JSON)

```
[
  {
    "id": "37f18971-1b18-4f52-aaaa-fe8b80c3523e",
    "title": "Insecure Deserialization in C:\\Users\\Dhruv\\Desktop\\CyberSceurity Project\\Test\\vuln_bank_lab\\app.py",
    "file": "C:\\Users\\Dhruv\\Desktop\\CyberSceurity Project\\Test\\vuln_bank_lab\\app.py",
    "line_start": 282,
    "line_end": 282,
    "severity": "HIGH",
    "vuln_type": "INSECURE_DESERIALIZATION",
    "description": "Detected the use of an insecure deserialization library in a Flask route. These libraries are vulnerable to deserialization attacks.",
    "rule_id": "python.flask.security.insecure-deserialization.insecure-deserialization",
    "confidence": 0.9,
    "cwe_id": "CWE-502: Deserialization of Untrusted Data",
    "cvss_score": null,
    "code_snippet": "
      280 |         try:
      281 |             decoded = base64.b64decode(payload)
      282 |             return jsonify(decoded)
    ",
    "sources": [
      "semgrep"
    ],
    "risk_score": 0.6076,
    "risk_category": "HIGH",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "The vulnerability in question arises from the use of pickle, an insecure deserialization library, in a Flask route. The code snippet shows a function that takes a payload and attempts to decode it using base64. If the decoding is successful, it returns the decoded payload as JSON. However, the code does not validate the decoded payload, which could be a maliciously crafted pickle object, leading to a deserialization attack.",
    "fix_suggestion": "Replace the use of pickle with a safer alternative such as JSON for serialization and deserialization.",
    "business_impact": "If exploited, an attacker could gain unauthorized access to sensitive user data or execute arbitrary code on the server.",
    "raw": {
      "check_id": "python.flask.security.insecure-deserialization.insecure-deserialization",
      "path": "C:\\Users\\Dhruv\\Desktop\\CyberSceurity Project\\Test\\vuln_bank_lab\\app.py",
      "start": {
        "line": 282,
        "col": 26,
        "offset": 9736
      },
      "end": {
        "line": 282,
        "col": 47,
        "offset": 9757
      },
      "extra": {

```

```

"message": "Detected the use of an insecure deserialization library in a Flask route. These libraries are insecure and can be exploited to execute arbitrary code or access sensitive data.",
"metadata": {
  "owasp": [
    "A08:2017 - Insecure Deserialization",
    "A08:2021 - Software and Data Integrity Failures",
    "A08:2025 - Software or Data Integrity Failures"
  ],
  "cwe": [
    "CWE-502: Deserialization of Untrusted Data"
  ],
  "references": [
    "https://docs.python.org/3/library/pickle.html"
  ],
  "category": "security",
  "technology": [
    "flask"
  ],
  "cwe2022-top25": true,
  "cwe2021-top25": true,
  "subcategory": [
    "audit"
  ],
  "likelihood": "LOW",
  "impact": "MEDIUM",
  "confidence": "LOW",
  "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
  "vulnerability_class": [
    "Insecure Deserialization "
  ],
  "source": "https://semgrep.dev/r/python.flask.security.insecure-deserialization.insecure-deserialization",
  "shortlink": "https://sg.run/N45z"
},
"severity": "ERROR",
"fingerprint": "requires login",
"lines": "requires login",
"validation_state": "NO_VALIDATOR",
"engine_kind": "OSS"
}
},
{
  "id": "27e52bb2-c9b8-47e2-b09d-89c9f139bf7e",
  "title": "Unknown in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py:70",
  "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
  "line_start": 70,
  "line_end": 70,
  "severity": "HIGH",
  "vuln_type": "UNKNOWN",
  "description": "Detected user input used to manually construct a SQL string. This is usually bad practice and can be exploited to execute arbitrary code or access sensitive data.",
  "rule_id": "python.flask.security.injection.tainted-sql-string.tainted-sql-string",
  "confidence": 0.9,
  "cwe_id": "CWE-704: Incorrect Type Conversion or Cast",
  "cvss_score": null,
  "code_snippet": "        68 |             email = request.form.get(\"email\", \"\")\n        69 |             passwo",
  "sources": [
    "semgrep"
  ],
  "risk_score": 0.539,
  "risk_category": "MEDIUM",
  "category_source": "score",
  "category_reason": "score-threshold",
  "llm_explanation": "The detected vulnerability in the provided code snippet involves the use of user input to manually construct a SQL string. This is a common vulnerability that can be exploited to execute arbitrary code or access sensitive data.",
  "fix_suggestion": "Replace the line 70 with:\n\nquery = \"SELECT * FROM users WHERE email = ? AND password = ?\"",
  "business_impact": "Exploiting this vulnerability could lead to unauthorized access of sensitive user data.",
  "raw": {
    "check_id": "python.flask.security.injection.tainted-sql-string.tainted-sql-string",
    "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
    "start": {
      "line": 70,
      "col": 17,
      "offset": 2086
    }
  }
}

```

```

    },
    "end": {
        "line": 70,
        "col": 91,
        "offset": 2160
    },
    "extra": {
        "message": "Detected user input used to manually construct a SQL string. This is usually bad practice",
        "metadata": {
            "cwe": [
                "CWE-704: Incorrect Type Conversion or Cast"
            ],
            "owasp": [
                "A01:2017 - Injection",
                "A03:2021 - Injection",
                "A05:2025 - Injection"
            ],
            "references": [
                "https://docs.sqlalchemy.org/en/14/core/tutorial.html#using-textual-sql",
                "https://www.tutorialspoint.com/sqlalchemy/sqlalchemy_quick_guide.htm",
                "https://docs.sqlalchemy.org/en/14/core/tutorial.html#using-more-specific-text-with-table-expressions"
            ],
            "category": "security",
            "technology": [
                "sqlalchemy",
                "flask"
            ],
            "subcategory": [
                "vuln"
            ],
            "impact": "MEDIUM",
            "likelihood": "MEDIUM",
            "confidence": "MEDIUM",
            "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
            "vulnerability_class": [
                "Improper Validation"
            ],
            "source": "https://semgrep.dev/r/python.flask.security.injection.tainted-sql-string.tainted-sql-string",
            "shortlink": "https://sg.run/JxZj"
        },
        "severity": "ERROR",
        "fingerprint": "requires login",
        "lines": "requires login",
        "validation_state": "NO_VALIDATOR",
        "engine_kind": "OSS"
    }
}
},
{
    "id": "a618b5d3-5b22-474d-b617-e28940c1c4b1",
    "title": "Sql Injection in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
    "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
    "line_start": 68,
    "line_end": 81,
    "severity": "MEDIUM",
    "vuln_type": "SQL_INJECTION",
    "description": "User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection.",
    "rule_id": "python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute",
    "confidence": 0.7,
    "cwe_id": "CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')",
    "cvss_score": null,
    "code_snippet": "
        66 |         error = None
        67 |         if request.method == \"POST\":
        68 |             email = request.form.get('email')
        69 |             password = request.form.get('password')
        70 |             if not email or not password:
        71 |                 error = \"Email and password are required\"
        72 |                 continue
        73 |             user = User.query.filter_by(email=email).first()
        74 |             if user and user.password == password:
        75 |                 session['user_id'] = user.id
        76 |                 return redirect(url_for('main.index'))
        77 |             error = \"Invalid email or password\"
        78 |     ",
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.517,
    "risk_category": "HIGH",
    "category_source": "vuln_floor",
    "category_reason": "minimum floor for SQL_INJECTION",
    "llm_explanation": "The vulnerability in question arises from the direct inclusion of user-controlled input into an SQL query without proper sanitization or escaping.",
    "fix_suggestion": "Replace line 68 with:\n\n```\npython\nemail = request.form.get('email')\npassword = request.form.get('password')\nif not email or not password:\n    error = 'Email and password are required'\n    continue\nuser = User.query.filter_by(email=email).first()\nif user and user.password == password:\n    session['user_id'] = user.id\n    return redirect(url_for('main.index'))\nerror = 'Invalid email or password'\n```"

```

```

"business_impact": "Exploiting this vulnerability could lead to significant financial loss and damage to the organization",
"raw": {
  "check_id": "python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection",
  "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
  "start": {
    "line": 68,
    "col": 9,
    "offset": 1978
  },
  "end": {
    "line": 81,
    "col": 51,
    "offset": 2624
  },
  "extra": {
    "message": "User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection vulnerability.",
    "metadata": {
      "cwe": [
        "CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')"
      ],
      "owasp": [
        "A01:2017 - Injection",
        "A03:2021 - Injection",
        "A05:2025 - Injection"
      ],
      "references": [
        "https://docs.djangoproject.com/en/3.0/topics/security/#sql-injection-protection"
      ],
      "category": "security",
      "technology": [
        "django"
      ],
      "cwe2022-top25": true,
      "cwe2021-top25": true,
      "subcategory": [
        "vuln"
      ],
      "likelihood": "MEDIUM",
      "impact": "HIGH",
      "confidence": "MEDIUM",
      "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
      "vulnerability_class": [
        "SQL Injection"
      ],
      "source": "https://semgrep.dev/r/python.django.security.injection.sql.sql-injection-using-db-cursor-execute",
      "shortlink": "https://sg.run/qx7y"
    },
    "severity": "WARNING",
    "fingerprint": "requires login",
    "lines": "requires login",
    "validation_state": "NO_VALIDATOR",
    "engine_kind": "OSS"
  }
},
{
  "id": "9cb4732e-17d3-42f9-8e46-4ebbd32cfb81",
  "title": "Xss in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\admin.html",
  "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\admin.html",
  "line_start": 22,
  "line_end": 22,
  "severity": "MEDIUM",
  "vuln_type": "XSS",
  "description": "Detected a segment of a Flask template where autoescaping is explicitly disabled with 'autoescape=False'. This could lead to a Cross-site Scripting (XSS) vulnerability.",
  "rule_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
  "confidence": 0.7,
  "cwe_id": "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
  "cvss_score": null,
  "code_snippet": "
    20 |         <button class='btn primary' type='submit'>Run</button>
    21 |     "
  "sources": [
    "semgrep"
  ]
}

```

```

],
"risk_score": 0.4747,
"risk_category": "MEDIUM",
"category_source": "score",
"category_reason": "score-threshold",
"llm_explanation": "The vulnerability in question arises from the use of the '| safe' filter in a Flask",
"fix_suggestion": "Remove the '| safe' filter to re-enable automatic HTML escaping for the content with",
"business_impact": "Exploiting this vulnerability could lead to unauthorized access to sensitive informa",
"raw": {
  "check_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
  "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\admin.html",
  "start": {
    "line": 22,
    "col": 26,
    "offset": 923
  },
  "end": {
    "line": 22,
    "col": 51,
    "offset": 948
  },
  "extra": {
    "message": "Detected a segment of a Flask template where autoescaping is explicitly disabled with ' ",
    "metadata": {
      "cwe": [
        "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
      ],
      "owasp": [
        "A07:2017 - Cross-Site Scripting (XSS)",
        "A03:2021 - Injection",
        "A05:2025 - Injection"
      ],
      "references": [
        "https://flask.palletsprojects.com/en/1.1.x/security/#cross-site-scripting-xss"
      ],
      "category": "security",
      "technology": [
        "flask"
      ],
      "cwe2022-top25": true,
      "cwe2021-top25": true,
      "subcategory": [
        "audit"
      ],
      "likelihood": "LOW",
      "impact": "MEDIUM",
      "confidence": "LOW",
      "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
      "vulnerability_class": [
        "Cross-Site-Scripting (XSS)"
      ],
      "source": "https://semgrep.dev/r/python.flask.security.xss.audit.template-unescaped-with-safe.tem",
      "shortlink": "https://sg.run/W8og"
    },
    "severity": "WARNING",
    "fingerprint": "requires login",
    "lines": "requires login",
    "validation_state": "NO_VALIDATOR",
    "engine_kind": "OSS"
  }
}
},
{
  "id": "61c9f490-5900-4b4a-b80c-b7917b7e348f",
  "title": "Unknown in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py:10",
  "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
  "line_start": 10,
  "line_end": 10,
  "severity": "HIGH",
  "vuln_type": "UNKNOWN",
  "description": "The Python documentation recommends using `defusedxml` instead of `xml` because the nat

```

```

"rule_id": "python.lang.security.use-defused-xml.use-defused-xml",
"confidence": 0.9,
"cwe_id": "CWE-611: Improper Restriction of XML External Entity Reference",
"cvss_score": null,
"code_snippet": "      8 | import subprocess\n      9 | import urllib.request\n>>> 10 | import xml.etree.ElementTree as ET\n\nsources": [\n    "semgrep"\n],\n"risk_score": 0.4508,\n"risk_category": "MEDIUM",\n"category_source": "score",\n"category_reason": "score-threshold",\n"llm_explanation": "The vulnerability in question arises from the use of the native Python `xml` library\n"fix_suggestion": "Replace the use of `xml.etree.ElementTree` with a more secure alternative such as `defusedxml`\n"business_impact": "If exploited, an attacker could gain unauthorized access to sensitive data within the application\n"raw": {\n    "check_id": "python.lang.security.use-defused-xml.use-defused-xml",\n    "path": "C:\\Users\\Dhruv\\Desktop\\CyberSecurity Project\\Test\\vuln_bank_lab\\app.py",\n    "start": {\n        "line": 10,\n        "col": 1,\n        "offset": 148\n    },\n    "end": {\n        "line": 10,\n        "col": 35,\n        "offset": 182\n    },\n    "extra": {\n        "message": "The Python documentation recommends using `defusedxml` instead of `xml` because the native\n        \"metadata\": {\n            \"owasp\": [\n                \"A04:2017 - XML External Entities (XXE)\",\n                \"A05:2021 - Security Misconfiguration\",\n                \"A02:2025 - Security Misconfiguration\"\n            ],\n            \"cwe\": [\n                \"CWE-611: Improper Restriction of XML External Entity Reference\"\n            ],\n            \"references\": [\n                \"https://docs.python.org/3/library/xml.html\",\n                \"https://github.com/tiran/defusedxml\",\n                \"https://owasp.org/www-community/vulnerabilities/XML_External_Entity_(XXE)_Processing\"\n            ],\n            \"category\": \"security\",\n            \"technology\": [\n                \"python\"\n            ],\n            \"cwe2022-top25\": true,\n            \"cwe2021-top25\": true,\n            \"subcategory\": [\n                \"audit\"\n            ],\n            \"likelihood\": \"LOW\",\n            \"impact\": \"MEDIUM\",\n            \"confidence\": \"LOW\",\n            \"license\": \"Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license\",\n            \"vulnerability_class\": [\n                \"XML Injection\"\n            ],\n            \"source\": \"https://semgrep.dev/r/python.lang.security.use-defused-xml.use-defused-xml\",\n            \"shortlink\": \"https://sg.run/kX47\"\n        },\n        \"severity\": \"ERROR\",\n        \"fingerprint\": \"requires login\",\n        \"lines\": \"requires login\",\n        \"validation_state\": \"NO_VALIDATOR\",\n        \"engine_kind\": \"OSS\"\n    }\n}\n},\n},

```

```

{
  "id": "c1d19dee-3b4f-4936-9ad8-eeaf53a90270",
  "title": "Open Redirect in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
  "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
  "line_start": 87,
  "line_end": 88,
  "severity": "HIGH",
  "vuln_type": "OPEN_REDIRECT",
  "description": "Data from request is passed to redirect(). This is an open redirect and could be exploited",
  "rule_id": "python.flask.security.open-redirect.open-redirect",
  "confidence": 0.9,
  "cwe_id": "CWE-601: URL Redirection to Untrusted Site ('Open Redirect')",
  "cvss_score": null,
  "code_snippet": "
      85 | @app.route(\"/logout\")\n
      86 | def logout():\n>>> 87 |     next_url = request.args.get('next', url_for('index'))\n
  "sources": [
    "semgrep"
  ],
  "risk_score": 0.4508,
  "risk_category": "MEDIUM",
  "category_source": "score",
  "category_reason": "score-threshold",
  "llm_explanation": "The vulnerability in question arises from the use of user input (specifically, the 'next' parameter) in the redirect() function without proper validation. This allows an attacker to redirect users to a malicious site.",
  "fix_suggestion": "Replace the line 87 with: \n\nnext_url = request.args.get('next', url_for('index'))\nif not next_url or not next_url.startswith('/'): \n    next_url = url_for('index')\nredirect(next_url)",
  "business_impact": "If exploited, an attacker could trick authenticated users into visiting a malicious website, potentially leading to data theft or account compromise.",
  "raw": {
    "check_id": "python.flask.security.open-redirect.open-redirect",
    "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
    "start": {
      "line": 87,
      "col": 5,
      "offset": 2756
    },
    "end": {
      "line": 88,
      "col": 34,
      "offset": 2844
    }
  },
  "extra": {
    "message": "Data from request is passed to redirect(). This is an open redirect and could be exploited",
    "metadata": {
      "cwe": [
        "CWE-601: URL Redirection to Untrusted Site ('Open Redirect')"
      ],
      "owasp": [
        "A01:2021 - Broken Access Control",
        "A01:2025 - Broken Access Control"
      ],
      "references": [
        "https://flask-login.readthedocs.io/en/latest/#login-example",
        "https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html",
        "https://docs.python.org/3/library/urllib.parse.html#url-parsing"
      ],
      "category": "security",
      "technology": [
        "flask"
      ],
      "subcategory": [
        "audit"
      ],
      "likelihood": "LOW",
      "impact": "MEDIUM",
      "confidence": "LOW",
      "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
      "vulnerability_class": [
        "Open Redirect"
      ],
      "source": "https://semgrep.dev/r/python.flask.security.open-redirect.open-redirect",
      "shortlink": "https://sg.run/kXe2"
    }
  },
  "severity": "ERROR",
  "fingerprint": "requires login",

```

```

        "lines": "requires login",
        "validation_state": "NO_VALIDATOR",
        "engine_kind": "OSS"
    }
}
},
{
    "id": "b293a4ee-fc39-4889-bbcf-e5e617a27961",
    "title": "Unknown in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py:313",
    "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
    "line_start": 313,
    "line_end": 313,
    "severity": "HIGH",
    "vuln_type": "UNKNOWN",
    "description": "Detected user input entering a `subprocess` call unsafely. This could result in a command execution.",
    "rule_id": "python.flask.security.injection.subprocess-injection.subprocess-injection",
    "confidence": 0.9,
    "cwe_id": "CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')",
    "cvss_score": null,
    "code_snippet": "
        311 |         cmd = request.args.get(\"cmd\", \"status\")\n
        312 |         shell_cmd =
    ",
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.4508,
    "risk_category": "MEDIUM",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "The vulnerability in question arises from the use of `subprocess.run` with `shell=True` and user input in the command string.",
    "fix_suggestion": "Replace line 313 with:\n\nproc = subprocess.run(['echo', 'Reconciled ledger for', cmd], shell=False)",
    "business_impact": "If exploited, an attacker could execute arbitrary commands on the server, leading to data loss or system compromise.",
    "raw": {
        "check_id": "python.flask.security.injection.subprocess-injection.subprocess-injection",
        "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
        "start": {
            "line": 313,
            "col": 12,
            "offset": 11098
        },
        "end": {
            "line": 313,
            "col": 81,
            "offset": 11167
        }
    },
    "extra": {
        "message": "Detected user input entering a `subprocess` call unsafely. This could result in a command execution.",
        "metadata": {
            "category": "security",
            "technology": [
                "flask"
            ],
            "owasp": [
                "A01:2017 - Injection",
                "A03:2021 - Injection",
                "A05:2025 - Injection"
            ],
            "cwe": [
                "CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')"
            ],
            "references": [
                "https://semgrep.dev/docs/cheat-sheets/python-command-injection/"
            ],
            "confidence": "HIGH",
            "cwe2022-top25": true,
            "cwe2021-top25": true,
            "subcategory": [
                "vuln"
            ],
            "likelihood": "HIGH",
            "impact": "MEDIUM",
            "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
            "vulnerability_class": [

```

```

        "Command Injection"
    ],
    "source": "https://semgrep.dev/r/python.flask.security.injection.subprocess-injection.subprocess-",
    "shortlink": "https://sg.run/5gw3"
},
{
    "severity": "ERROR",
    "fingerprint": "requires login",
    "lines": "requires login",
    "validation_state": "NO_VALIDATOR",
    "engine_kind": "OSS"
}
}
},
{
    "id": "238bae73-a0d0-4b23-bf9e-8e62db1ec1a9",
    "title": "Xss in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\account.html",
    "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\account.html",
    "line_start": 24,
    "line_end": 24,
    "severity": "MEDIUM",
    "vuln_type": "XSS",
    "description": "Detected a segment of a Flask template where autoescaping is explicitly disabled with 'safe' filter.",
    "rule_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
    "confidence": 0.7,
    "cwe_id": "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
    "cvss_score": null,
    "code_snippet": "
                22 |         {% for tx in transactions %}
                23 |             <tr>
                24 |
    ",
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.4042,
    "risk_category": "MEDIUM",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "The vulnerability in question arises from the use of the 'safe' filter within a Flask template. The 'safe' filter is used to escape HTML special characters, but it is not applied to the 'tx' variable in the provided code snippet, which is iterated over in a loop. This allows an attacker to inject malicious HTML or JavaScript code into the rendered output.",
    "fix_suggestion": "Replace 'safe' with 'escape' to ensure HTML autoescaping is enabled again. The 'escape' filter will automatically escape any HTML special characters in the output, preventing an attacker from injecting malicious code.",
    "business_impact": "If exploited, an attacker could execute arbitrary JavaScript on the client side, leading to session hijacking, data theft, or other malicious actions.",
    "raw": {
        "check_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
        "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\account.html",
        "start": {
            "line": 24,
            "col": 13,
            "offset": 667
        },
        "end": {
            "line": 24,
            "col": 39,
            "offset": 693
        }
    },
    "extra": {
        "message": "Detected a segment of a Flask template where autoescaping is explicitly disabled with 'safe' filter.",
        "metadata": {
            "cwe": [
                "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')"
            ],
            "owasp": [
                "A07:2017 - Cross-Site Scripting (XSS)",
                "A03:2021 - Injection",
                "A05:2025 - Injection"
            ],
            "references": [
                "https://flask.palletsprojects.com/en/1.1.x/security/#cross-site-scripting-xss"
            ],
            "category": "security",
            "technology": [
                "flask"
            ],
            "cwe2022-top25": true,
            "cwe2021-top25": true,
            "subcategory": [

```

```

        "audit"
    ],
    "likelihood": "LOW",
    "impact": "MEDIUM",
    "confidence": "LOW",
    "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
    "vulnerability_class": [
        "Cross-Site-Scripting (XSS)"
    ],
    "source": "https://semgrep.dev/r/python.flask.security.xss.audit.template-unescaped-with-safe.tem",
    "shortlink": "https://sg.run/W8og"
},
{
    "severity": "WARNING",
    "fingerprint": "requires login",
    "lines": "requires login",
    "validation_state": "NO_VALIDATOR",
    "engine_kind": "OSS"
}
}
},
{
    "id": "fd5410b1-1c18-4f4c-af2e-d9ef2e738e88",
    "title": "Xss in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\dashb",
    "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\dashboard.htm",
    "line_start": 61,
    "line_end": 61,
    "severity": "MEDIUM",
    "vuln_type": "XSS",
    "description": "Detected a segment of a Flask template where autoescaping is explicitly disabled with '
    "rule_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
    "confidence": 0.7,
    "cwe_id": "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
    "cvss_score": null,
    "code_snippet": "
        59 |
        <td>{{ tx.counterparty }}</td>\n
        60 |
        <td>\u20b9{{
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.4042,
    "risk_category": "MEDIUM",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "The detected vulnerability in the Flask template at line 61 is a cross-site scripting
    "fix_suggestion": "Remove the '|' safe' filter to enable automatic escaping of HTML characters within the
    "business_impact": "Exploiting this vulnerability could lead to unauthorized access to sensitive user da
    "raw": {
        "check_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
        "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\dashboard.f
        "start": {
            "line": 61,
            "col": 15,
            "offset": 2039
        },
        "end": {
            "line": 61,
            "col": 33,
            "offset": 2057
        },
        "extra": {
            "message": "Detected a segment of a Flask template where autoescaping is explicitly disabled with '
            "metadata": {
                "cwe": [
                    "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
                ],
                "owasp": [
                    "A07:2017 - Cross-Site Scripting (XSS)",
                    "A03:2021 - Injection",
                    "A05:2025 - Injection"
                ],
                "references": [
                    "https://flask.palletsprojects.com/en/1.1.x/security/#cross-site-scripting-xss"
                ]
            }
        }
    },
    "references": [
        "https://flask.palletsprojects.com/en/1.1.x/security/#cross-site-scripting-xss"
    ]
},

```

```

    "category": "security",
    "technology": [
      "flask"
    ],
    "cwe2022-top25": true,
    "cwe2021-top25": true,
    "subcategory": [
      "audit"
    ],
    "likelihood": "LOW",
    "impact": "MEDIUM",
    "confidence": "LOW",
    "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
    "vulnerability_class": [
      "Cross-Site-Scripting (XSS)"
    ],
    "source": "https://semgrep.dev/r/python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
    "shortlink": "https://sg.run/W8og"
  },
  "severity": "WARNING",
  "fingerprint": "requires login",
  "lines": "requires login",
  "validation_state": "NO_VALIDATOR",
  "engine_kind": "OSS"
}
},
{
  "id": "059daaf2-e71b-4b35-bb89-053861908095",
  "title": "Xss in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\reflect.html",
  "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\reflect.html",
  "line_start": 7,
  "line_end": 7,
  "severity": "MEDIUM",
  "vuln_type": "XSS",
  "description": "Detected a segment of a Flask template where autoescaping is explicitly disabled with 'safe' in the affected template.",
  "rule_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
  "confidence": 0.7,
  "cwe_id": "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
  "cvss_score": null,
  "code_snippet": "5 | <h1>Customer Notice Board</h1>\n6 | <p class=\"subtle\">This page is under construction.\n\n</p>\n\n</div>\n\n</html>\n\n",
  "sources": [
    "semgrep"
  ],
  "risk_score": 0.4042,
  "risk_category": "MEDIUM",
  "category_source": "score",
  "category_reason": "score-threshold",
  "llm_explanation": "The vulnerability in the provided Flask template snippet is a cross-site scripting (XSS) attack. The snippet shows a Flask template where autoescaping is disabled with 'safe' in the affected template. This allows an attacker to inject malicious HTML or JavaScript code into the page. The fix suggestion is to replace 'safe' with a default HTML escaping filter like '<| e' in the affected template. Exploiting this vulnerability could lead to unauthorized access to sensitive user data.",
  "fix_suggestion": "Replace 'safe' with a default HTML escaping filter like '<| e' in the affected template.",
  "business_impact": "Exploiting this vulnerability could lead to unauthorized access to sensitive user data.",
  "raw": {
    "check_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
    "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\reflect.html",
    "start": {
      "line": 7,
      "col": 28,
      "offset": 255
    },
    "end": {
      "line": 7,
      "col": 43,
      "offset": 270
    }
  },
  "extra": {
    "message": "Detected a segment of a Flask template where autoescaping is explicitly disabled with 'safe' in the affected template.",
    "metadata": {
      "cwe": [
        "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
        "OWASP: Cross-Site Scripting (XSS)"
      ],
      "owasp": [
        "XSS"
      ]
    }
  }
}
}

```

```

        "A07:2017 - Cross-Site Scripting (XSS)",
        "A03:2021 - Injection",
        "A05:2025 - Injection"
    ],
    "references": [
        "https://flask.palletsprojects.com/en/1.1.x/security/#cross-site-scripting-xss"
    ],
    "category": "security",
    "technology": [
        "flask"
    ],
    "cwe2022-top25": true,
    "cwe2021-top25": true,
    "subcategory": [
        "audit"
    ],
    "likelihood": "LOW",
    "impact": "MEDIUM",
    "confidence": "LOW",
    "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
    "vulnerability_class": [
        "Cross-Site-Scripting (XSS)"
    ],
    "source": "https://semgrep.dev/r/python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
    "shortlink": "https://sg.run/W8og"
},
{
    "severity": "WARNING",
    "fingerprint": "requires login",
    "lines": "requires login",
    "validation_state": "NO_VALIDATOR",
    "engine_kind": "OSS"
}
},
{
    "id": "15d330e0-06af-4340-9666-66e85ab115a6",
    "title": "Xss in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\support.html",
    "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\support.html",
    "line_start": 23,
    "line_end": 23,
    "severity": "MEDIUM",
    "vuln_type": "XSS",
    "description": "Detected a segment of a Flask template where autoescaping is explicitly disabled with 'safe' filter.",
    "rule_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
    "confidence": 0.7,
    "cwe_id": "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
    "cvss_score": null,
    "code_snippet": "
21 |         <td>{{ ticket.name }}</td>\n
22 |         <td>{{ ticket.subject }}</td>\n
",
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.4042,
    "risk_category": "MEDIUM",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "The vulnerability in question is a cross-site scripting (XSS) flaw present in the Flask template engine. It arises from the use of the 'safe' filter in the Jinja2 template, which does not escape special characters like '<' and '>'. This allows an attacker to inject malicious HTML or JavaScript code into the rendered output.",
    "fix_suggestion": "Remove or replace the 'safe' filter with a proper escaping mechanism to ensure that user input is correctly escaped before being rendered in the browser.",
    "business_impact": "Exploiting this vulnerability could lead to unauthorized access to sensitive information, defacement of the application, or execution of arbitrary code on the user's browser.",
    "raw": {
        "check_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
        "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\support.html",
        "start": {
            "line": 23,
            "col": 15,
            "offset": 853
        },
        "end": {
            "line": 23,
            "col": 37,
            "offset": 875
        }
    },
    "end": {
        "line": 23,
        "col": 37,
        "offset": 875
    }
},

```

```

"extra": {
  "message": "Detected a segment of a Flask template where autoescaping is explicitly disabled with '
  "metadata": {
    "cwe": [
      "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
    ],
    "owasp": [
      "A07:2017 - Cross-Site Scripting (XSS)",
      "A03:2021 - Injection",
      "A05:2025 - Injection"
    ],
    "references": [
      "https://flask.palletsprojects.com/en/1.1.x/security/#cross-site-scripting-xss"
    ],
    "category": "security",
    "technology": [
      "flask"
    ],
    "cwe2022-top25": true,
    "cwe2021-top25": true,
    "subcategory": [
      "audit"
    ],
    "likelihood": "LOW",
    "impact": "MEDIUM",
    "confidence": "LOW",
    "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
    "vulnerability_class": [
      "Cross-Site-Scripting (XSS)"
    ],
    "source": "https://semgrep.dev/r/python.flask.security.xss.audit.template-unescaped-with-safe.tem
    "shortlink": "https://sg.run/W8og"
  },
  "severity": "WARNING",
  "fingerprint": "requires login",
  "lines": "requires login",
  "validation_state": "NO_VALIDATOR",
  "engine_kind": "OSS"
}
}
},
{
  "id": "2057f2ee-b5fa-405c-ba8a-70115ace8943",
  "title": "Unknown in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py:248",
  "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
  "line_start": 248,
  "line_end": 248,
  "severity": "MEDIUM",
  "vuln_type": "UNKNOWN",
  "description": "Detected a dynamic value being used with urllib. urllib supports 'file://' schemes, so a
  "rule_id": "python.lang.security.audit.dynamic-urllib-use-detected.dynamic-urllib-use-detected",
  "confidence": 0.7,
  "cwe_id": "CWE-939: Improper Authorization in Handler for Custom URL Scheme",
  "cvss_score": null,
  "code_snippet": "
      246 | def fetch_remote():\n
      247 |     target = request.args.get(\"url\", \"ht
  "sources": [
    "semgrep"
  ],
  "risk_score": 0.3384,
  "risk_category": "LOW",
  "category_source": "score",
  "category_reason": "score-threshold",
  "llm_explanation": "The vulnerability in question arises from the use of user-controlled input (specifici
  "fix_suggestion": "Replace 'urllib.request.urlopen' with a safer method that validates and sanitizes use
  "business_impact": "If exploited, this vulnerability could lead to unauthorized access to sensitive file
  "raw": {
    "check_id": "python.lang.security.audit.dynamic-urllib-use-detected.dynamic-urllib-use-detected",
    "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
    "start": {
      "line": 248,
      "col": 10,

```

```

        "offset": 8376
    },
    "end": {
        "line": 248,
        "col": 51,
        "offset": 8417
    },
    "extra": {
        "message": "Detected a dynamic value being used with urllib. urllib supports 'file://\' schemes, so a",
        "metadata": {
            "cwe": [
                "CWE-939: Improper Authorization in Handler for Custom URL Scheme"
            ],
            "owasp": "A01:2017 - Injection",
            "source-rule-url": "https://github.com/PyCQA/bandit/blob/b1411bfb43795d3ffd268bef17a839dee954c2b1",
            "bandit-code": "B310",
            "asvs": {
                "control_id": "5.2.4 Dynamic Code Execution Features",
                "control_url": "https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-Sanitization",
                "section": "V5: Validation, Sanitization and Encoding Verification Requirements",
                "version": "4"
            },
            "category": "security",
            "technology": [
                "python"
            ],
            "references": [
                "https://cwe.mitre.org/data/definitions/939.html"
            ],
            "subcategory": [
                "audit"
            ],
            "likelihood": "LOW",
            "impact": "LOW",
            "confidence": "LOW",
            "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
            "vulnerability_class": [
                "Improper Authorization"
            ],
            "source": "https://semgrep.dev/r/python.lang.security.audit.dynamic-urllib-use-detected.dynamic-u",
            "shortlink": "https://sg.run/dkZZ"
        },
        "severity": "WARNING",
        "fingerprint": "requires login",
        "lines": "requires login",
        "validation_state": "NO_VALIDATOR",
        "engine_kind": "OSS"
    }
}
},
{
    "id": "f9c6b7e9-ee80-4b56-bd6b-a8d7bc8fdbb4",
    "title": "Unknown in C:\\Users\\Dhruv\\Desktop\\CyberSecurity Project\\Test\\vuln_bank_lab\\app.py:369",
    "file": "C:\\Users\\Dhruv\\Desktop\\CyberSecurity Project\\Test\\vuln_bank_lab\\app.py",
    "line_start": 369,
    "line_end": 369,
    "severity": "MEDIUM",
    "vuln_type": "UNKNOWN",
    "description": "Detected Flask app with debug=True. Do not deploy to production with this flag enabled a",
    "rule_id": "python.flask.security.audit.debug-enabled.debug-enabled",
    "confidence": 0.7,
    "cwe_id": "CWE-489: Active Debug Code",
    "cvss_score": null,
    "code_snippet": "
        367 | \n
        368 | if __name__ == \"__main__\":\n>>>
        369 |     app.run(host=\"127.0.0.1\", port=5000)
    ",
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.282,
    "risk_category": "LOW",
    "category_source": "score",
    "category_reason": "score-threshold",

```

```

"llm_explanation": "The detected vulnerability in the Flask application involves running the app with the
"fix_suggestion": "Replace line 369 with: \napp.run(host='127.0.0.1', port=5001)",
"business_impact": "Exploiting this vulnerability could lead to unauthorized access and exposure of sensitive data",
"raw": {
  "check_id": "python.flask.security.audit.debug-enabled.debug-enabled",
  "path": "C:\\Users\\Dhruv\\Desktop\\CyberSecurity Project\\Test\\vuln_bank_lab\\app.py",
  "start": {
    "line": 369,
    "col": 5,
    "offset": 12789
  },
  "end": {
    "line": 369,
    "col": 53,
    "offset": 12837
  },
  "extra": {
    "message": "Detected Flask app with debug=True. Do not deploy to production with this flag enabled",
    "metadata": {
      "cwe": [
        "CWE-489: Active Debug Code"
      ],
      "owasp": "A06:2017 - Security Misconfiguration",
      "references": [
        "https://labs.detectify.com/2015/10/02/how-patreon-got-hacked-publicly-exposed-werkzeug-debugger/"
      ],
      "category": "security",
      "technology": [
        "flask"
      ],
      "subcategory": [
        "vuln"
      ],
      "likelihood": "HIGH",
      "impact": "MEDIUM",
      "confidence": "HIGH",
      "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
      "vulnerability_class": [
        "Active Debug Code"
      ],
      "source": "https://semgrep.dev/r/python.flask.security.audit.debug-enabled.debug-enabled",
      "shortlink": "https://sg.run/dKrd"
    },
    "severity": "WARNING",
    "fingerprint": "requires login",
    "lines": "requires login",
    "validation_state": "NO_VALIDATOR",
    "engine_kind": "OSS"
  }
}
}
]

```